Recovery

What you will learn from this lecture:

1. Introduction
2. Transactions
3. Potential problems
4. Method of Recovery

1. Introduction

Recovery(恢复) in a database system means, primarily, recovering the database itself: that is, restoring the database to a state that is known to be correct or consistent after some failure has rendered the current state inconsistent, or at least suspect.

The underlying principles of recovery is: redundancy(冗余).

2. Transactions

A transaction is a logical unit of work.

```
    BEGIN TRANSACTION ;

    INSERT INTO SP
           RELATION { TUPLE { S#   S#   ( 'S5' ),
                              P#   P#   ( 'P1' ),
                              QTY  QTY  ( 1000 ) } } ;
    IF any error occurred THEN GO TO UNDO ; END IF ;

    UPDATE P WHERE P# = P# ( 'P1' )
           TOTQTY := TOTQTY + QTY ( 1000 ) ;
    IF any error occurred THEN GO TO UNDO ; END IF ;

    COMMIT ;
    GO TO FINISH ;

UNDO :
    ROLLBACK ;

FINISH :
    RETURN ;
```

2. Transactions…

The **COMMIT** operation signals successful end-of-transaction: It tells the transaction manager that a logical unit of work has been successfully completed, the database is (or should be) in a consistent state again, and all of the updates made by that unit of work can now be "committed" or made permanent.

The **ROLLBACK** operation signals unsuccessful end-of-transaction: It tells the transaction manager that something has gone wrong, the database might be in an inconsistent state, and all of the updates made by the logical unit of work so far must be "rolled back" or undone.

2. Transactions-commit point

COMMIT establishes a commit point.

A commit point thus corresponds to the end of a logical unit of work, and hence to a point at which the database is or should be in a consistent state.

ROLLBACK, by contrast, rolls the database back to the state it was in at BEGIN TRANSACTION, which effectively means back to the previous commit point.

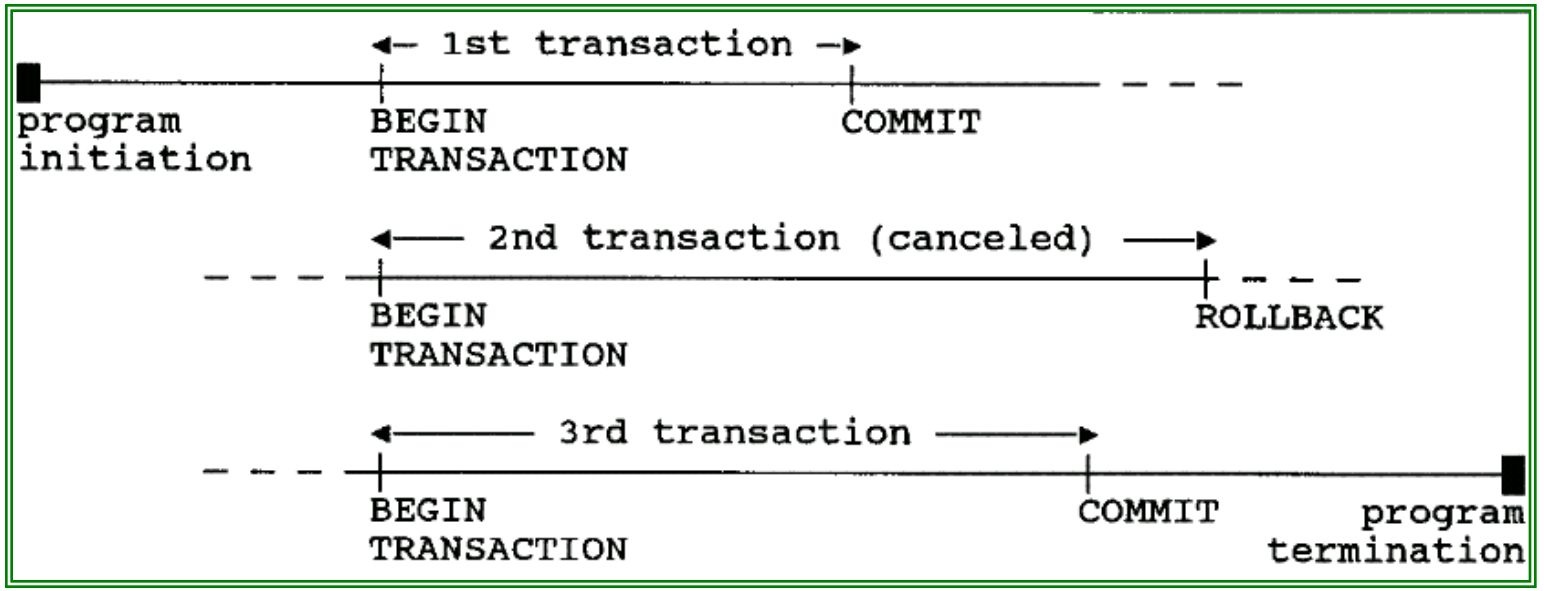
2. Transactions-commit point

When a commit point is established:

- All updates made by the executing program since the previous commit point are committed; that is, they are made permanent.
- All database positioning is lost and all tuple locks are released.

2. Transactions-commit point

Note: COMMIT and ROLLBACK terminate the transaction, not the program.



2. Transactions-commit point

The system **cannot** assume that application programs will always include explicit tests for all possible errors.

Therefore, the system will issue an implicit ROLLBACK for any transaction that fails for any reason to reach its planned termination.

"planned termination" means either an explicit COMMIT or an explicit ROLLBACK.

2. Transaction-ACID Properties

Atomicity(原子性): Transactions are atomic (all or nothing).

Consistency(一致性): Transactions preserve database consistency.

- a transaction transforms a consistent state of the database into another consistent state, without necessarily preserving consistency at all intermediate points.

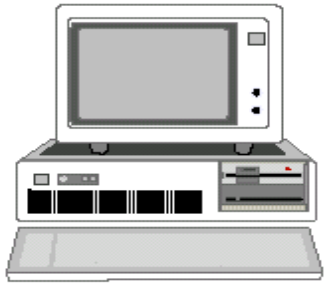
2. Transaction-ACID Properties

Isolation(隔离性): Transactions are isolated from one another.

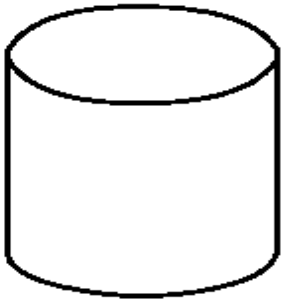
- Even though in general there will be many transactions running concurrently, any given transaction's updates are concealed from all the rest, until that transaction commits.
- For any two distinct transactions T1 and T2, T1 might see T2's updates (after T2 has committed) or T2 might see T1's updates (after T1 has committed), but certainly not both.

Durability(持久性): Once a transaction commits, its updates survive in the database, even if there is a subsequent system crash.

3. Potential problems

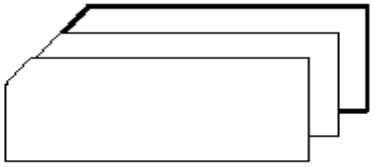


HARDWARE

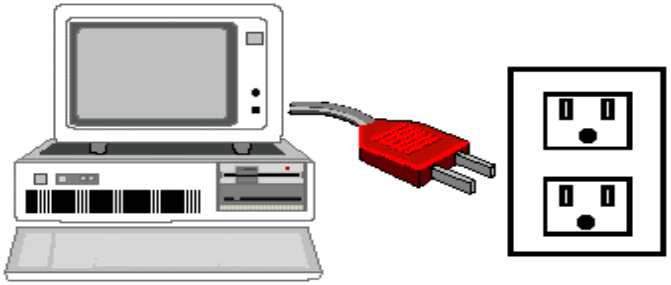


MEDIA

OPERATIONAL



SOFTWARE



POWER

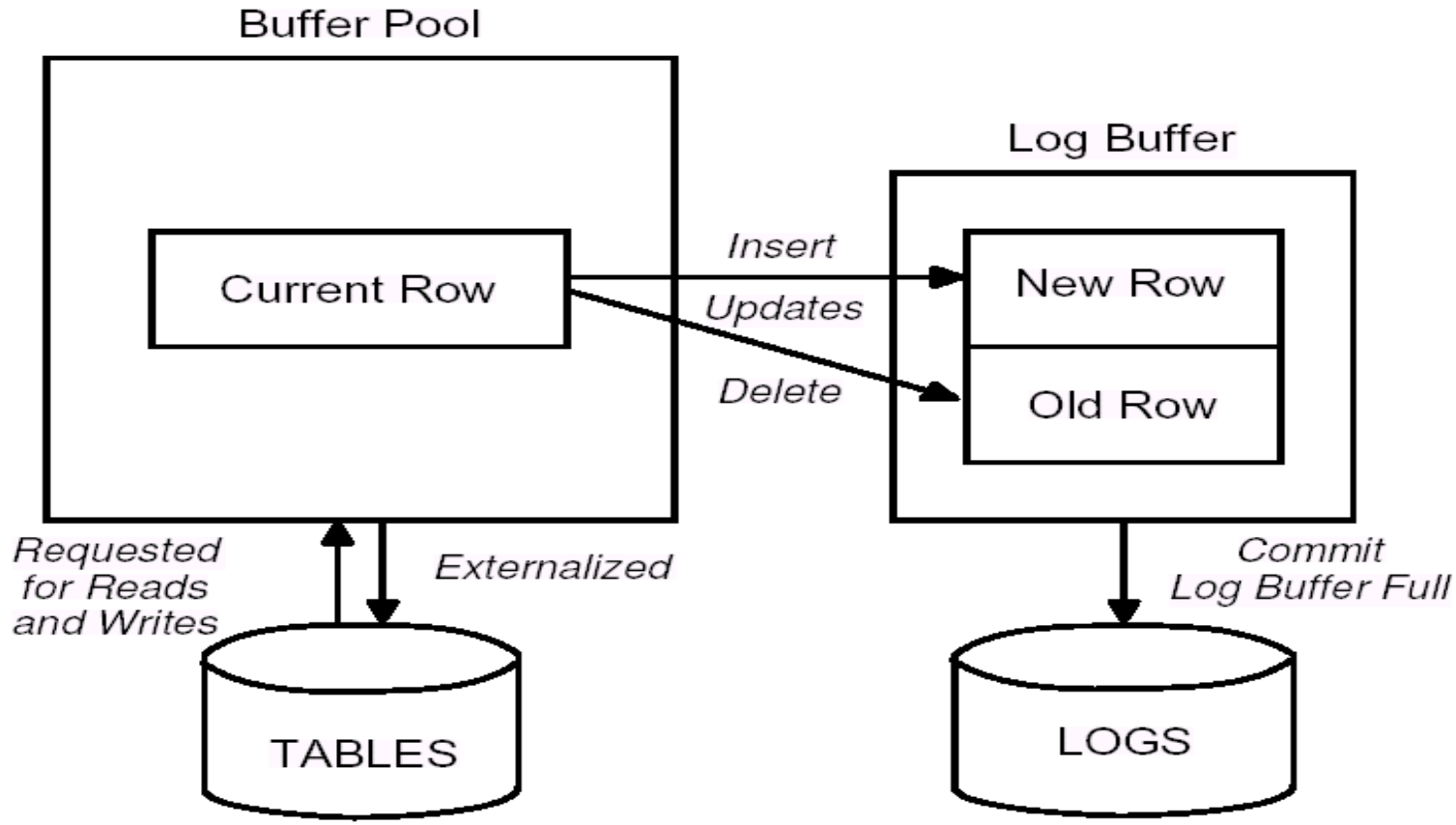
3. Potential problems

Transaction failures: Transaction fails for any reason to reach its planned termination.

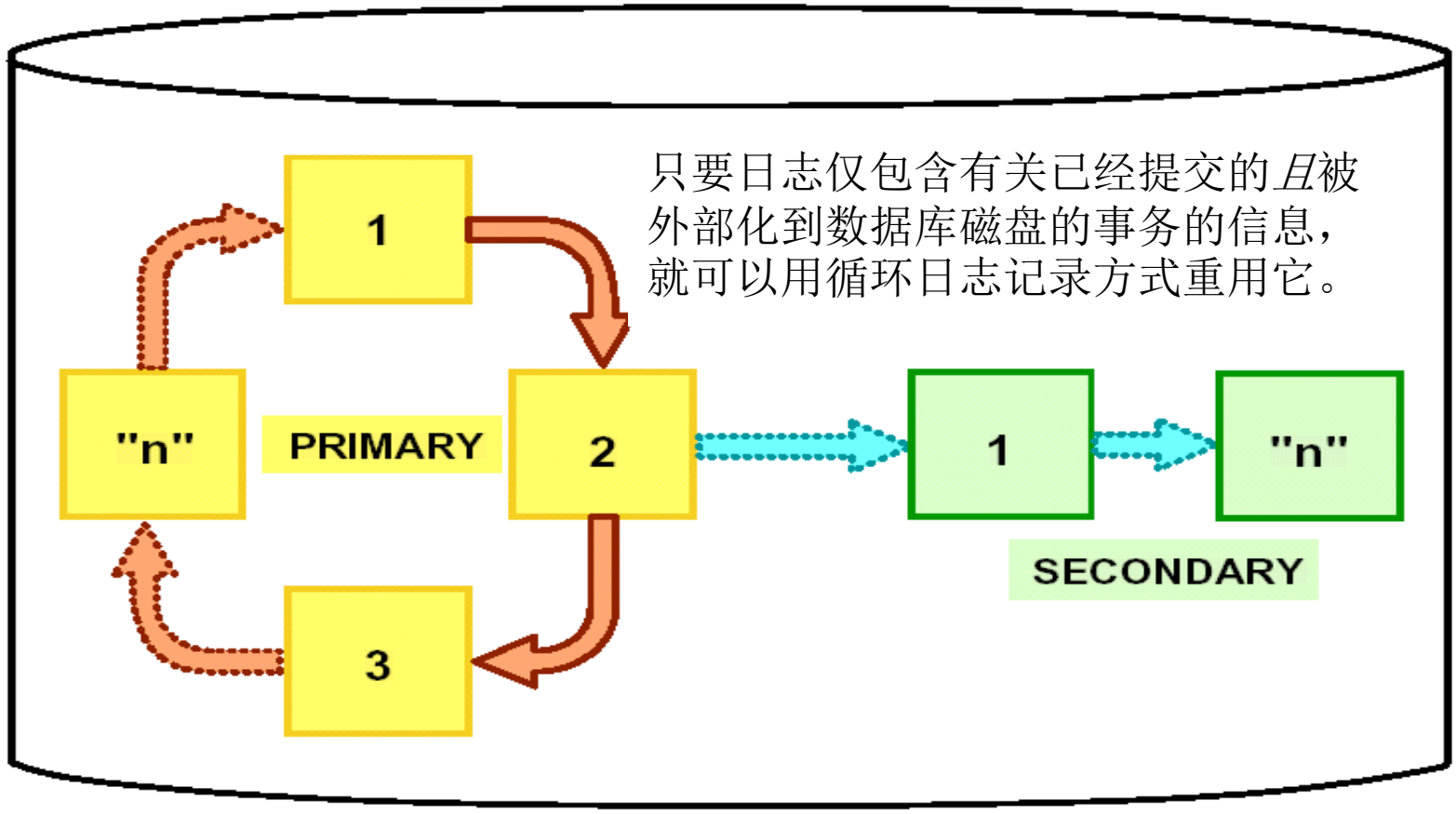
System failures: (e.g., power outage) Affect all transactions currently in progress but do not physically damage the database. A system failure is sometimes called a soft crash.

Media failures: Cause damage to the database, or to some portion of it, and affect at least those transactions currently using that portion. A media failure is sometimes called a hard crash.

Logging Introduction



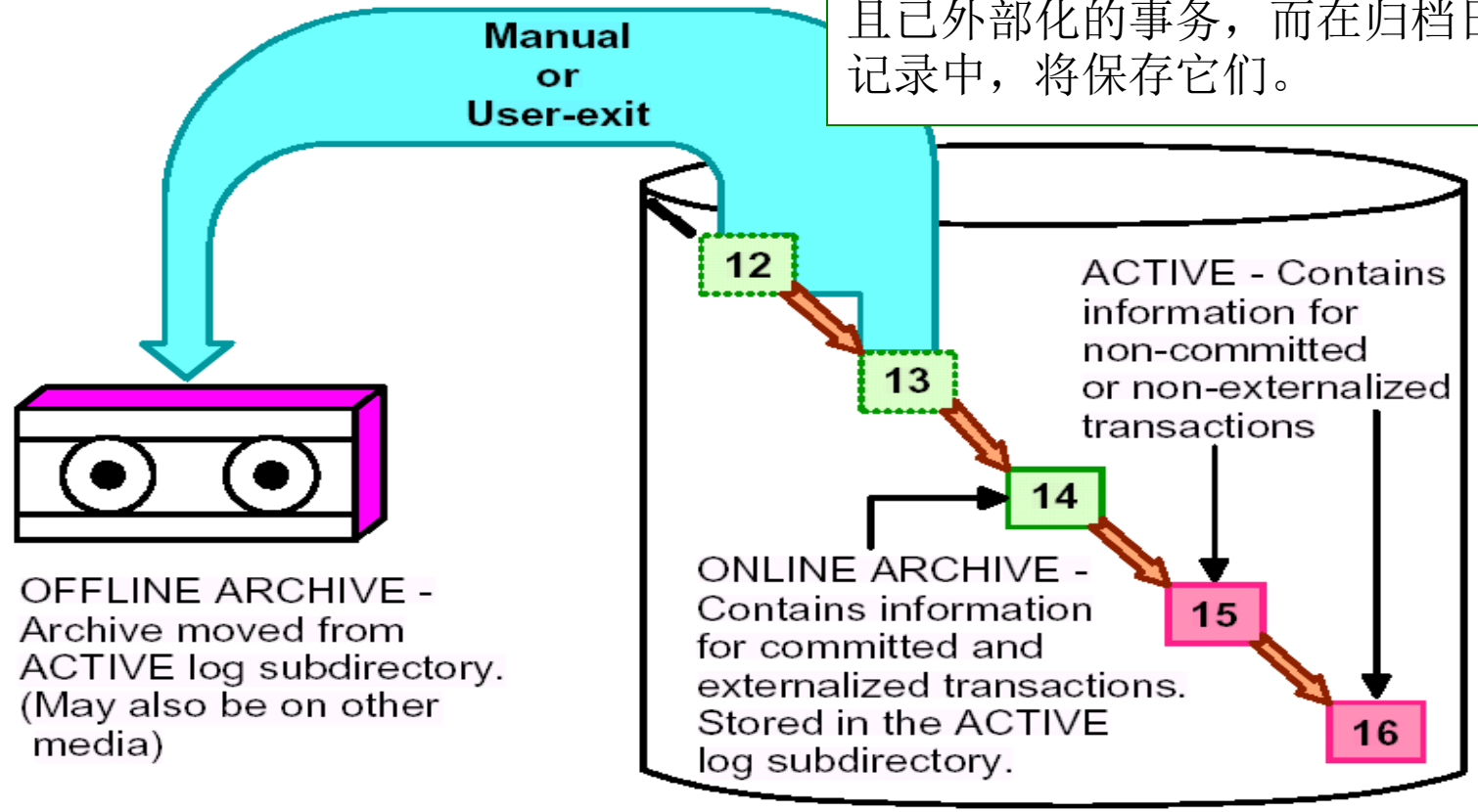
Circular logging(循环日志)



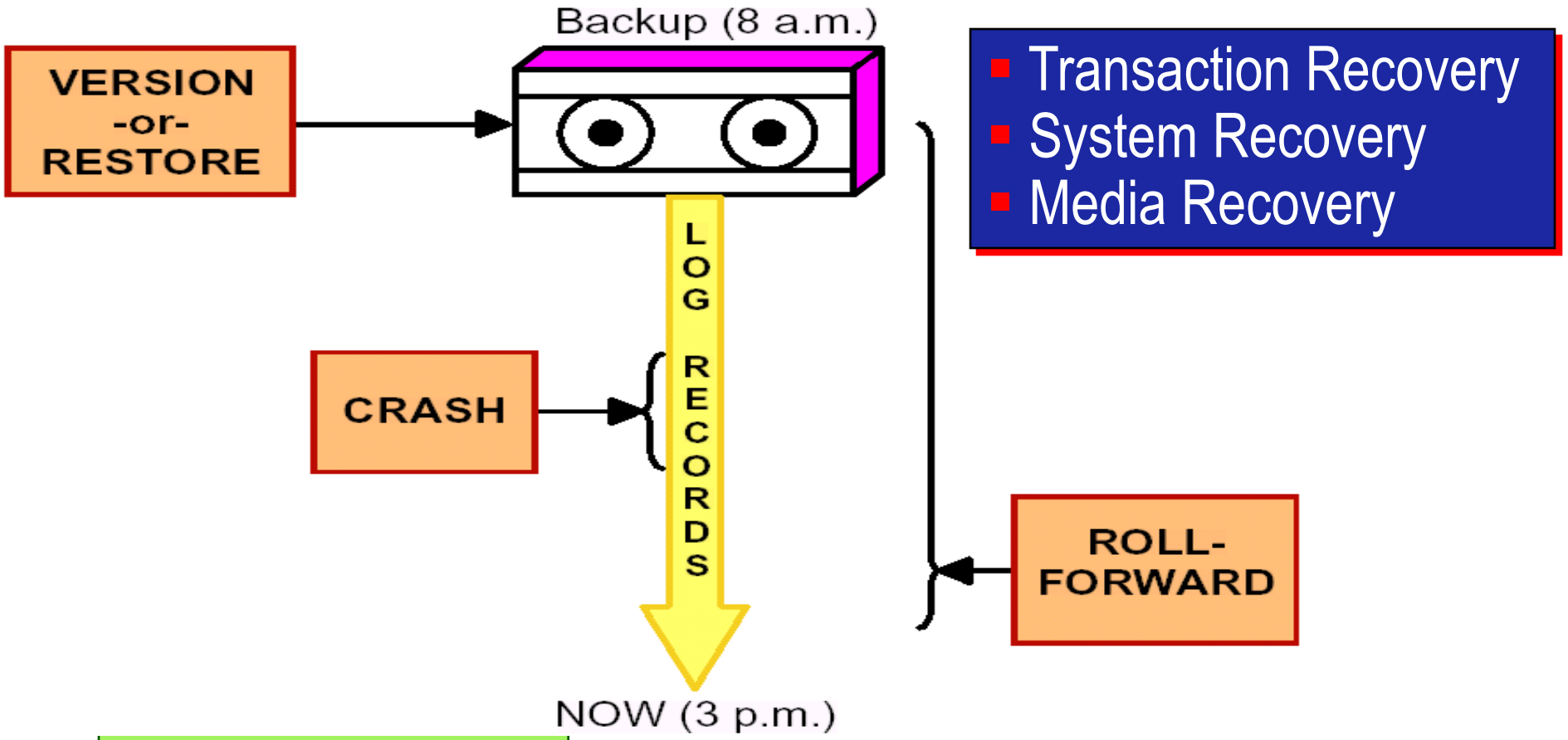
只要日志仅包含有关已经提交的*旦*被外部化到数据库磁盘的事务的信息，就可以用循环日志记录方式重用它。

Archival logging(归档日志)

在循环日志记录中，要覆盖已提交且已外部化的事务，而在归档日志记录中，将保存它们。



4. Mothod of Recovery



- Transaction Recovery
- System Recovery
- Media Recovery

Transaction Recovery

If a transaction successfully commits, then the system will guarantee that its updates will be permanently installed in the database, even if the system crashes the very next moment.

The system might crash after the COMMIT has been honored but before the updates have been physically written to the database—they might still be waiting in a main memory buffer and so be lost at the time of the crash.

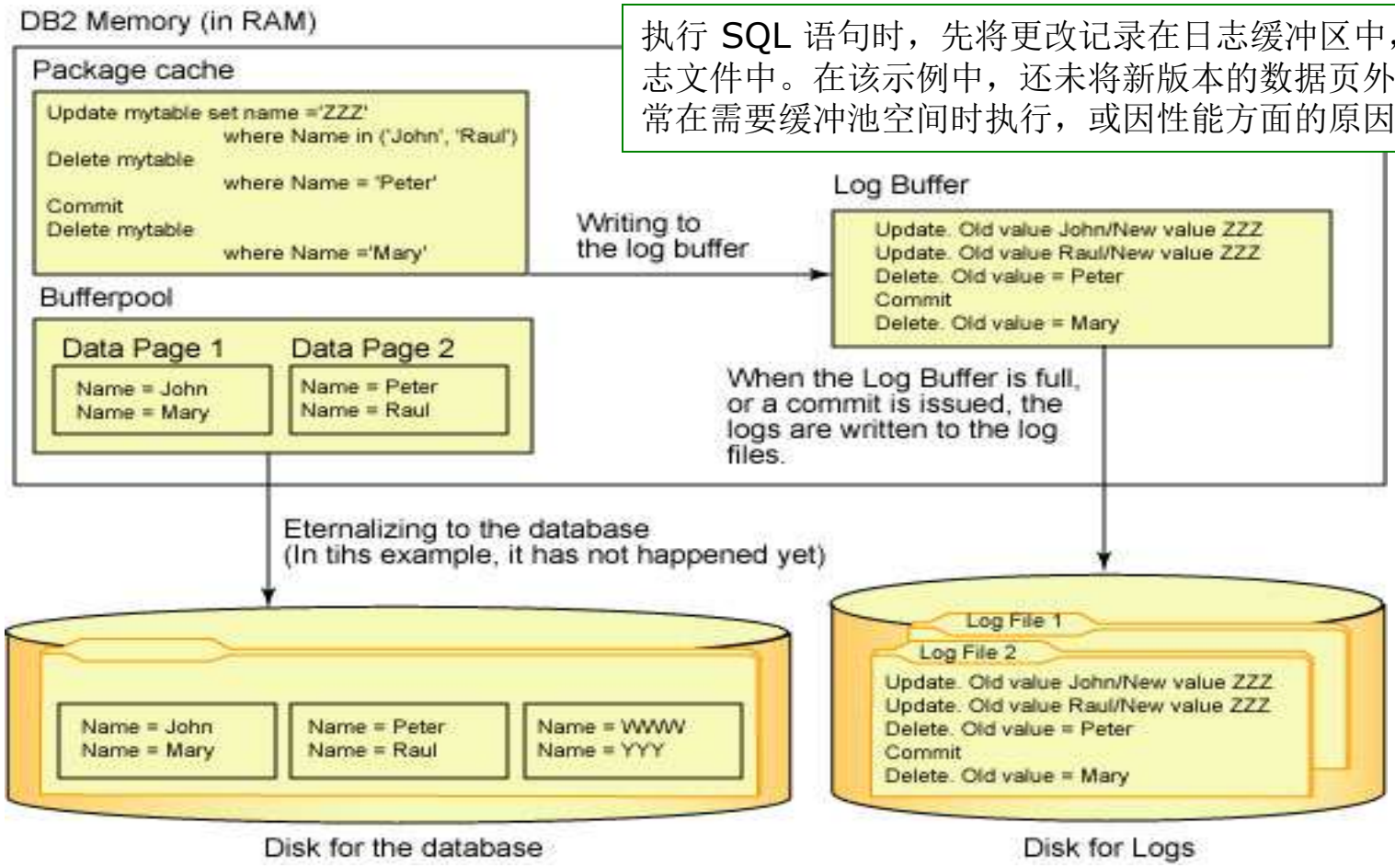
Transaction Recovery

Even if that happens, the system's restart procedure will still install those updates in the database; it is able to discover the values to be written by examining the relevant entries in the log.

It follows that the log must be physically written before COMMIT processing can complete—the write-ahead log rule.

The write-ahead log rule

执行 SQL 语句时，先将更改记录在日志缓冲区中，然后将更改写入日志文件中。在该示例中，还未将新版本的数据页外部化到数据库；这通常在需要缓冲池空间时执行，或因性能方面的原因而异步执行。



System Recovery

Any transaction which was in progress at the time of the system failure must be undone at restart time.

Any transaction that did successfully complete prior to the crash but did not manage to get their updates transferred from the database buffers to the physical database must be redo at restart time.

System Recovery

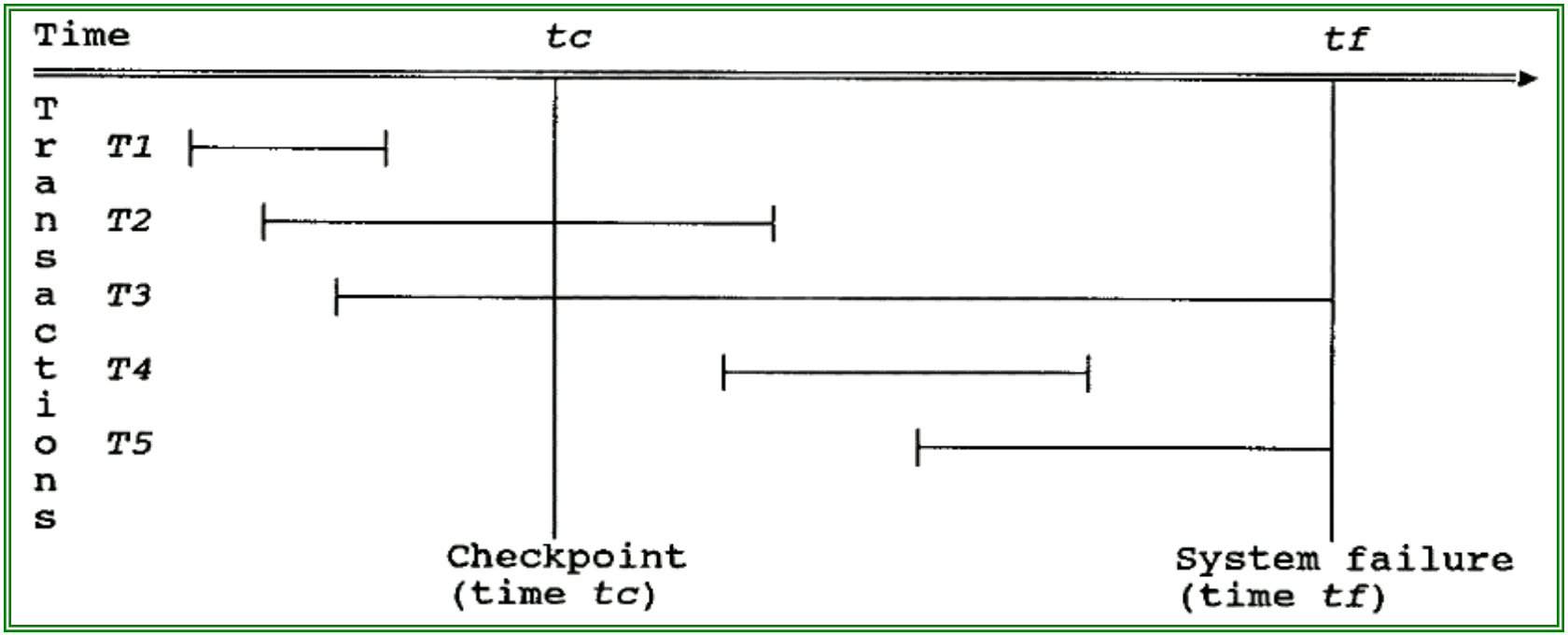
How does the system know at restart time which transactions to undo and which to redo?

At certain prescribed intervals — typically whenever some prescribed number of entries have been written to the log— the system automatically takes a checkpoint.

Taking a checkpoint involves:

- physically writing ("force-writing") the contents of the database buffers out to the physical database.
- physically writing a special checkpoint record out to the physical log. The checkpoint record gives a list of all transactions that were in progress at the time the checkpoint was taken. (eg. T2,T3)

System Recovery



Undo: T3, T5

Redone: T2, T4

System Recovery
Identify the UNDO list and REDO list

1. Start with two lists of transactions, the UNDO list and the REDO list. Set the UNDO list equal to the list of all transactions given in the most recent checkpoint record; set the REDO list to empty.
2. Search forward through the log, starting from the checkpoint record.
3. If a BEGIN TRANSACTION log entry is found for transaction T, add T to the UNDO list. (eg. T4)
4. If a COMMIT log entry is found for transaction T, move T from the UNDO list to the REDO list.
5. When the end of the log is reached, the UNDO and REDO lists identify, respectively, transactions of types T3 and T5 and transactions of types T2 and T4.

System Recovery

The system now works backward through the log, undoing the transactions in the UNDO list; then it works forward again, redoing the transactions in the REDO list.

Restoring the database to a consistent state by undoing work is sometimes called backward recovery(后滚恢复).

Restoring the database to a consistent state by redoing work is sometimes called forward recovery(前滚恢复).

Media Recovery
BACKUP DATABASE

BACKUP DATABASE sample

TO d:\mybackups

BACKUP DATABASE sample ONLINE

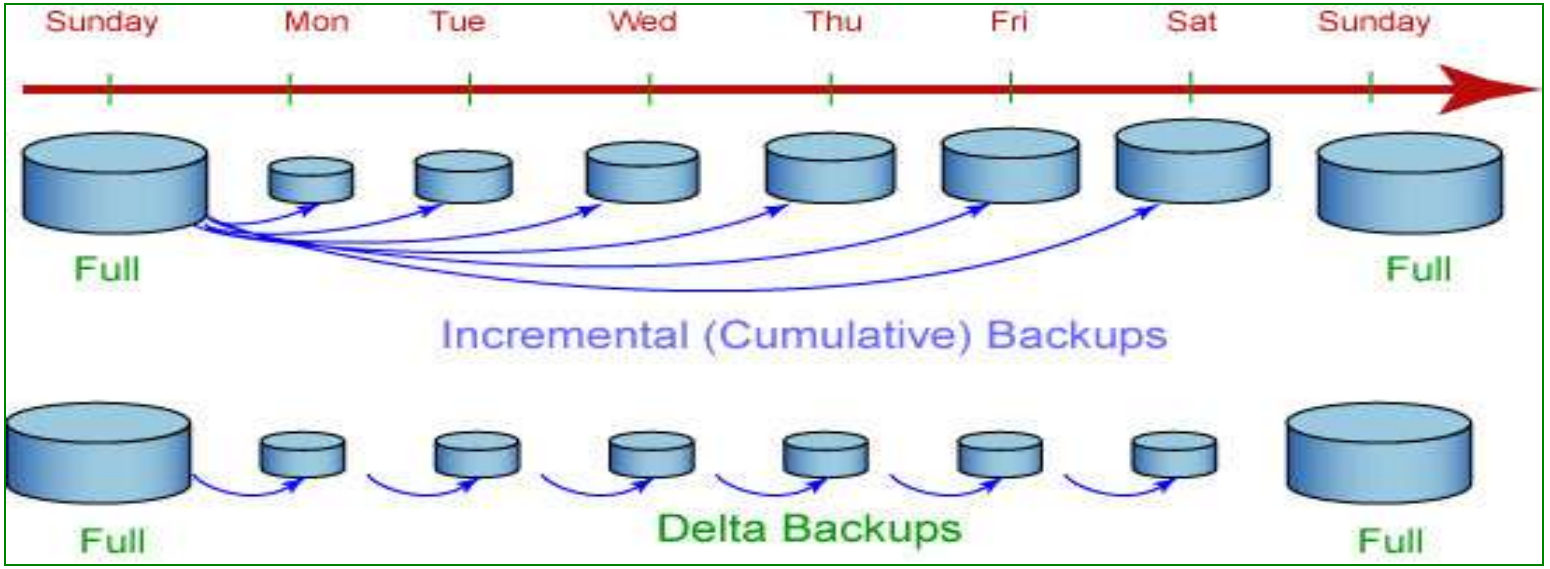
TO /dev/rdir1, /dev/rdir2

联机备份允许其他用户在备份数据库时对它进行访问。这些用户所做的一些更改很可能在备份时没有存储在备份副本中。因此，恢复时需要联机备份和一组完整的归档日志。

增量备份 (eg.DB2)

增量：DB2 备份自上次完全数据库备份以来所更改的所有数据。

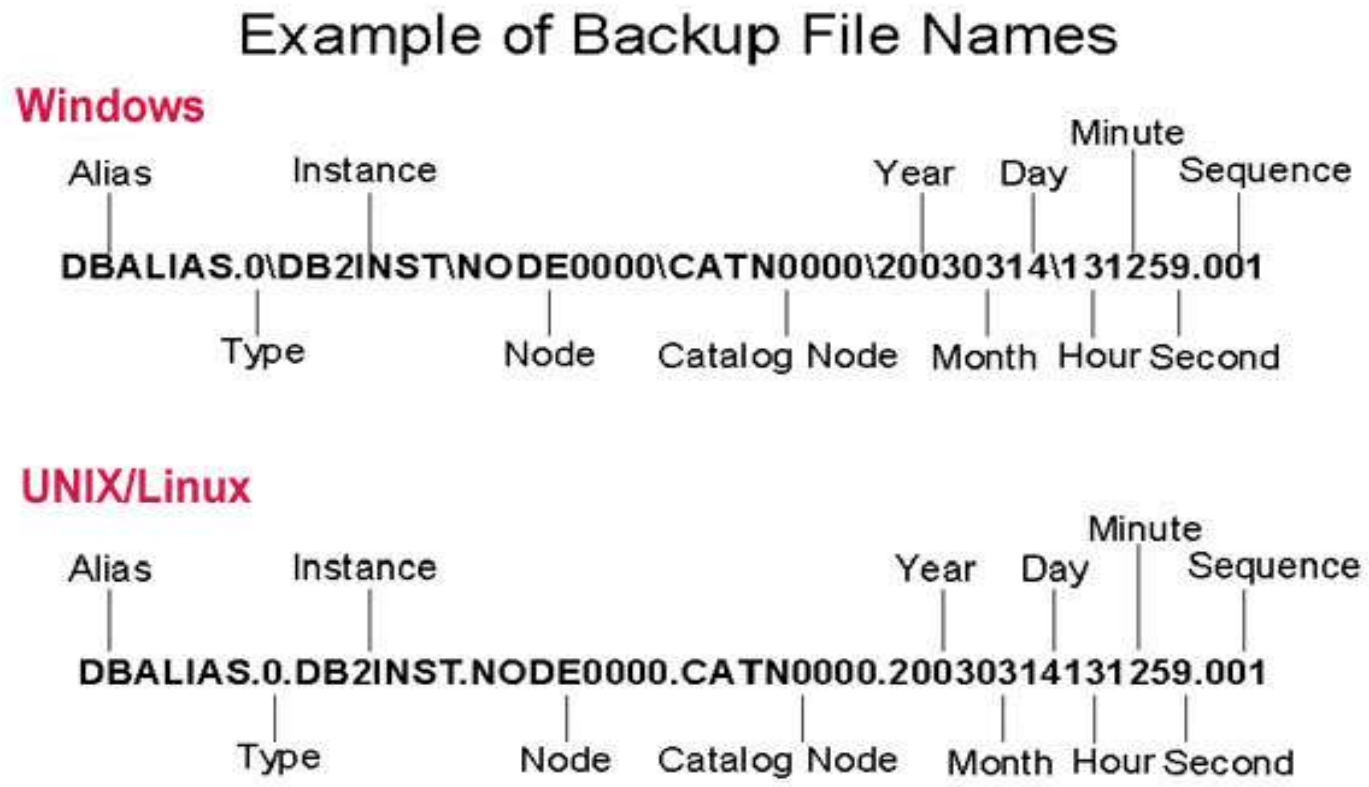
delta：DB2 将只备份自上一次成功的完全、增量或差异备份以来所更改的数据。



如果星期五进行增量备份之后发生崩溃，那么可以先恢复第一个星期日的完全备份，然后恢复星期五进行的增量备份。

如果星期五进行 delta 备份之后发生崩溃，那么可以先恢复第一个星期日的完全备份，然后恢复从星期一到星期五（包括星期五）所进行的每次 delta 备份。

Backup file Names of DB2



Restore Database

```
RESTORE DATABASE sample
FROM  d:\mybackups
TAKEN AT 20030314131259
```

Thank You!